

RFU - Reserved for Future use [unused]

```
#define LED_GREEN 3
```

-1- LED - BLUE 17

$$(volatile \text{ uint32_t } *) \text{GPIO_out} =$$
$$(volatile int 32_t *) (GPIO_Base | GPIO_out_offset)$$

Ein *qp GPIO_out |= (1 << LED_GREEN | 1 << LED_BLUE);

AUS *gp GPIDont Q = ~ (1 << LED - ... - (1 -) ;

```
Toggle: *gp GPIDout 1 = (1 << LED ... );
```

Zugriffe

```
#define GPIO_OUT (*(volatile uint32_t*)(BASE | GPIO_OUT))
```

```
GPIOOUT |= (1 << LED_BLUE);
```

Bitfield:

struct x {

$\text{uint } 32_t \text{ } a; \leftarrow \text{Komponente}$

```
uint32_t b;
```

3.

```

struct GPIOOut {
    uint32_t rfu1:3;
    uint32_t LED_GREEN:1;
    uint32_t rfu2:3;
    uint32_t LED_BLUE:1;
    uint32_t rfu3:24;
}

```

```

struct TEST {
    (uint16_t a:12;
    uint16_t b:2;
    - uint16_t c:8;
    - uint32_t d:4;
}

```

```

struct TEST test;

```

12:a	4:b	8:c	8:rfu	4:d	28:rfu
------	-----	-----	-------	-----	--------

#define ON 1

#define OFF 0

```

(volatile struct GPIOOut*) gp GPIOOut =

```

```

(volatile struct GPIOOut*)(GPIOBASE + GPIOOut_OFFSET);

```

```

gp GPIOOut -> LED_GREEN = ON;

```

```

gp GPIOOut -> LED_BLUE = ON;

```

(sat) RGB HTML
00 00 00 00 [0x] (color # 400040)

```
struct RGB {  
    uint8_t blue;  
    uint8_t green;  
    uint8_t red;  
    uint8_t rfu;  
};
```

```
struct RGB c = { blue = 0x40, green = 0, red = 0x40 };  
uint32_t k = *(uint32_t*) &c;
```

```
union pixel {  
    struct RGB rgb;  
    uint32_t raw;  
};
```

```
union pixel px {0};  
px.rgb.red = 0x40;  
px.rgb.blue = 0x40;  
printf ("%X", px.raw);
```

```

union GPIOData {
    struct { anonym
        uint32_t rfu1:3;
        uint32_t ledGreen:1;
        uint32_t rfu2:3;
        uint32_t ledBlue:1;
    };
    uint32_t raw;
}

```

```

(volatile union GPIO*) gpGPIO =
    (volatile union GPIO*) (GPIOBASE1 GPIO_OUT_OFFSET);

```

```

gpGPIO->ledGreen = 0;

```

```

gpGPIO->raw = (1 << LED_GREEN) | (1 << LED_BLUE);

```

LED-HUE